Vulnerability to Victory: Mastering the CVE Lifecycle for Java Developers

📸 slides and references 📸

🙋‍♂️ Anthony Dahanne, Software Developer @Herodevs

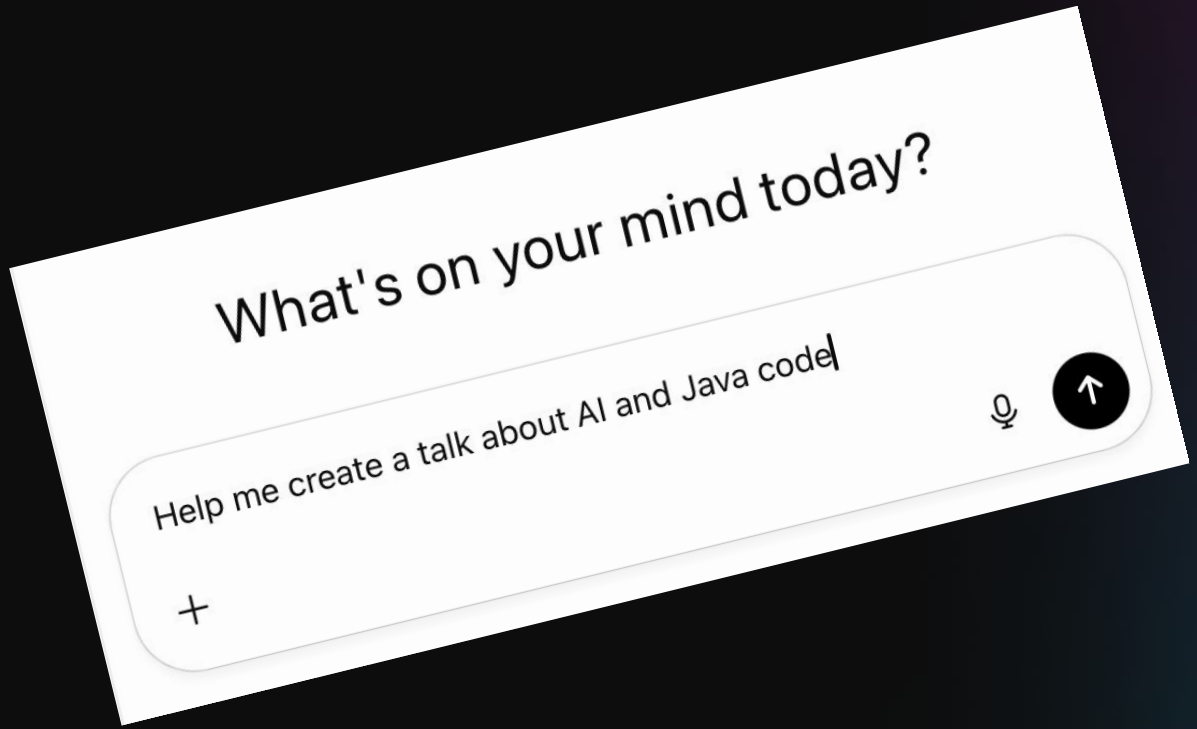
@anthony.dahannet.net

framapiaf.org/@anthonydahanne

📅 February 25th 2026

📍 Confoo

What do you expect from this talk?

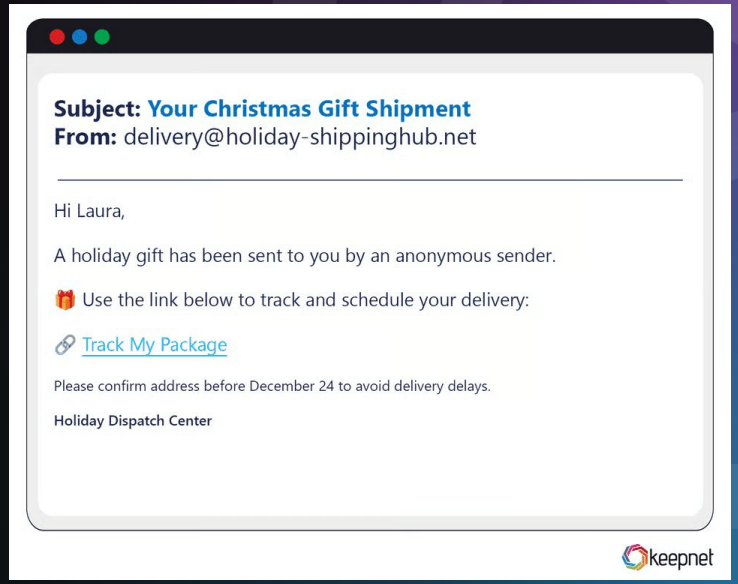


AI?

🤷

You already know about security

Don’t let people enter the building with your badge!



Do not click on phishing email
(even if they promise you nice gifts!)

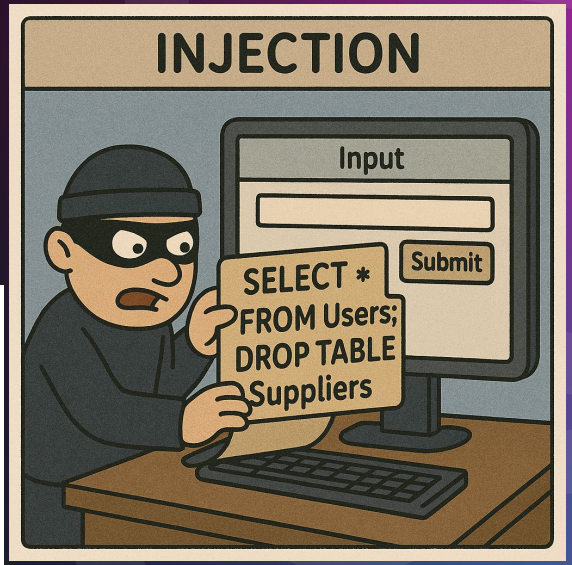
… but a software developer has more responsibilities

OWASP®

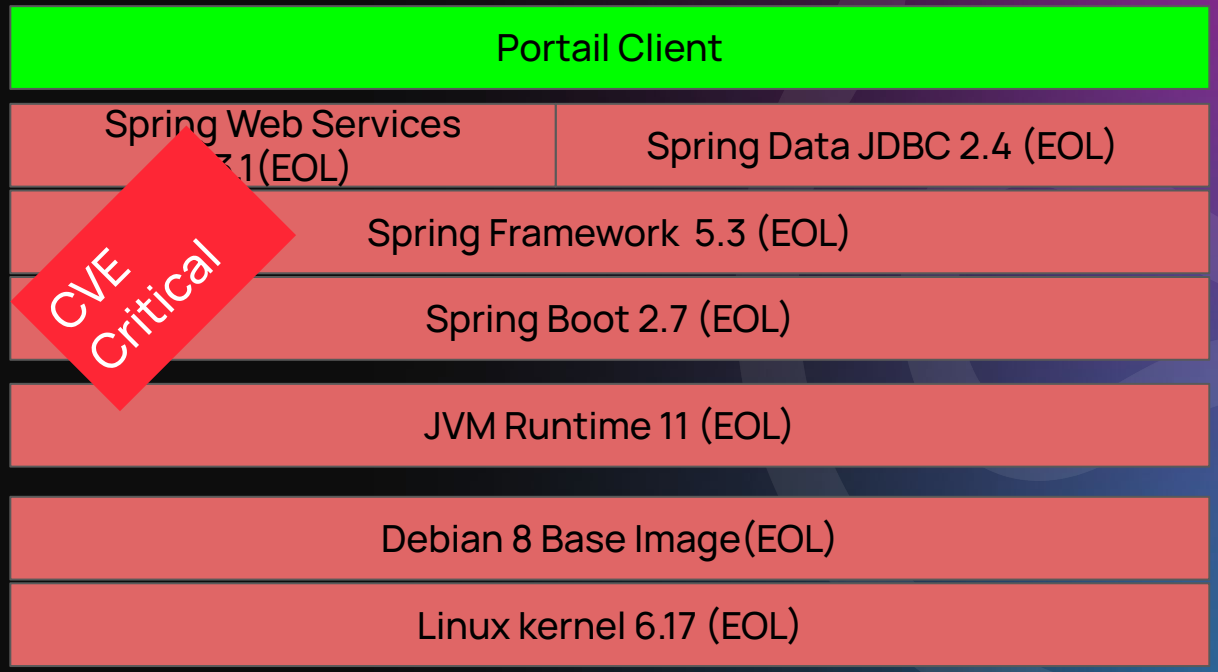
Open Web Application Security Project

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures



… but a software developer has more responsibilities



EOL: End of Life - no more updates

… but a software developer has more responsibilities

Overview | Components

Audit Vulnerabilities 135 133

Apply VEX | Export VEX

	Component
>	h2
>	h2
>	h2
>	spring-boot
>	spring-boot-actuator
>	spring-boot-actuator-autoconfigure
>	spring-boot-autoconfig

CVE-2023-34040 Detail

Description

In Spring for Apache Kafka 3.0.9 and earlier and versions 2.9.10 and earlier, a possible deserialization attack vector existed, but only if unusual configuration was applied. An attacker would have to construct a malicious serialized object in one of the deserialization exception record headers. Specifically, an application is vulnerable when all of the following are true: * The user does not configure an ErrorHandlingDeserializer for the key and/or value of the record * The user explicitly sets container properties checkDeserExWhenKeyNull and/or checkDeserExWhenValueNull container properties to true. * The user allows untrusted sources to publish to a Kafka topic By default, these properties are false, and the container only attempts to deserialize the headers if an ErrorHandlingDeserializer is configured. The ErrorHandlingDeserializer prevents the vulnerability by removing any such malicious headers before processing the record.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

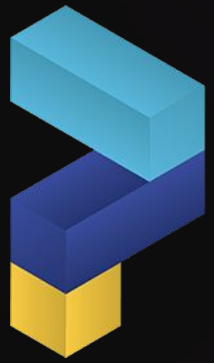
CVSS 3.x Severity and Vector Strings:

NIST: NVD	**Base Score:** 7.8 HIGH	**Vector:** CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CNA: VMware	**Base Score:** 5.3 MEDIUM	**Vector:** CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:L

Your presenter for this session

herodevs

☕Java and Go developer, Cloud Architect, Devops, etc. but also community leader 🫶



paketo
buildpacks

Effortless Containerization with Cloud Native Buildpacks

February 25, 2026 @ 15:00

Westmount 6

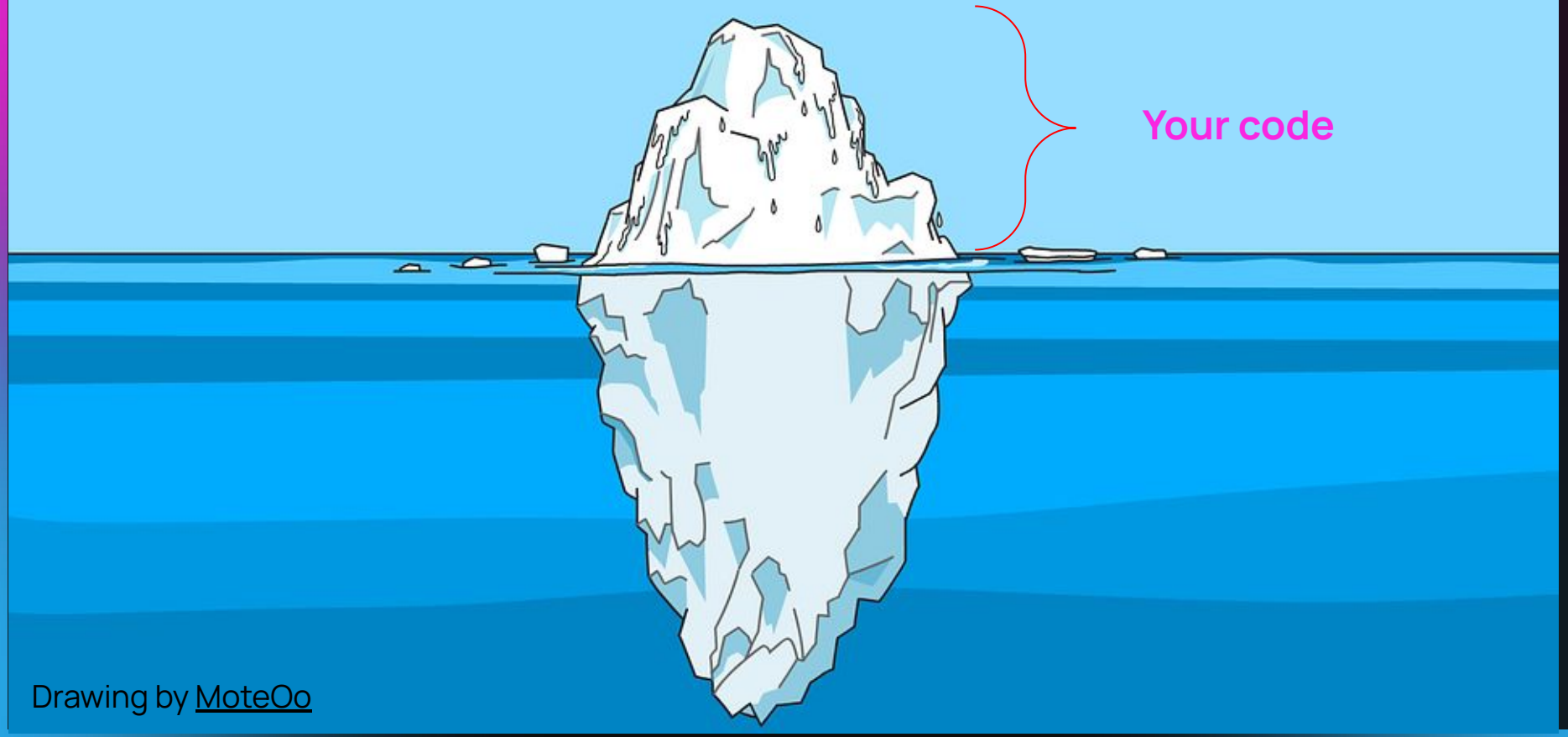
Cloud Computing | DevOps

Non Profit Orgs. President 👩‍💼

Agenda

- Static application security testing (SAST)
- Listing your dependencies (SBOM)
- Matching your SBOM with known vulnerabilities (SCA)
  - Operating Systems (OS) Dependencies vulnerabilities
  - Java Dependencies vulnerabilities
- Towards 0 CVEs (Common Vulnerabilities and Exposures)

What you ship to production

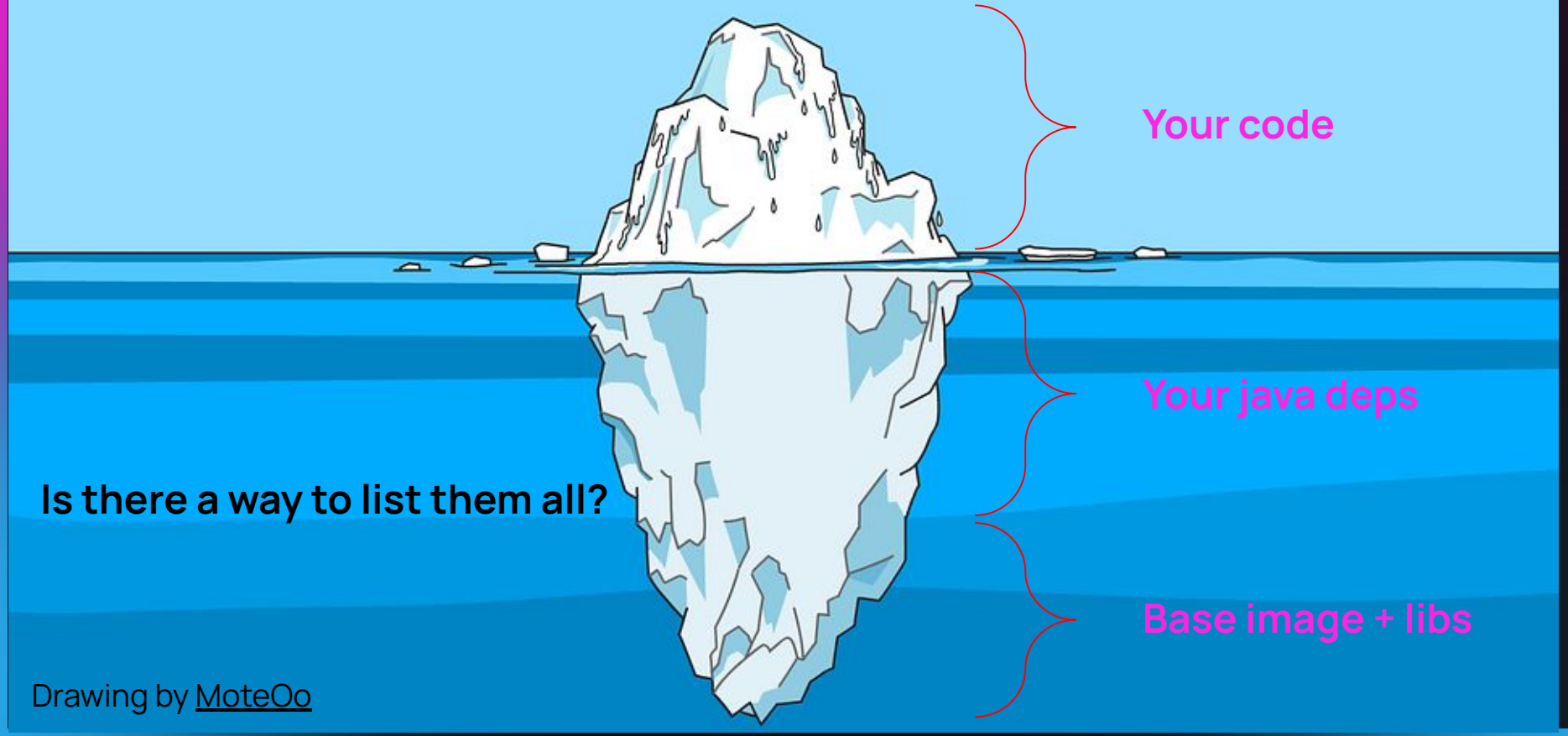


Drawing by MoteOo

Static application security testing (SAST)

- From within your ide
  - FindSecurityBugs (maven / gradle plugin and local UI)
  - Qodana from Jetbrains
- From CI; same list as before but we can add
  - SonarQube
  - Gitlab SAST (based on SemGrep)
- There are many others though, mainly commercial tools: Veracode, Checkmarx, Mend, etc.

What you ship to production



Is there a way to list them all?

Drawing by MoteOo

What exactly goes to prod?

- Shipping containers? What are the OS libs you included?
- You have to know ALL what is running in prod - pom? Build.gradle? MANIFEST.MF ?
- The industry has settled on Software Bill of Materials (SBOMs)
  - Of course, there are several formats 🫠
  - SPDX: a bit older, great for licenses
  - CycloneDX: the industry favorite

What's an SBOM like?

```
{
   "type" : "library",
   "bom-ref" : "pkg:maven/org.ehcache/ehcache@3.10.8?type=jar",
   "publisher" : "Terracotta Inc.",
   "group" : "org.ehcache",
   "name" : "ehcache",
   "version" : "3.10.8",
   "description" : "End-user ehcache3 jar artifact",
   "licenses" : [
     {
       "license" : {
         "id" : "Apache-2.0"
       }
     }
   ],
   "purl" : "pkg:maven/org.ehcache/ehcache@3.10.8?type=jar",
   "externalReferences" : [
     {
       "type" : "website",
       "url" : "http://ehcache.org"
     }      ]
 },
```

publisher

group, name, version

description / summary

license (using SPDX License id)

purl

website

Side note: pURL vs CPE

- Package URL:
  - `pkg:maven/org.springframework.cloud/spring-cloud-bindings@2.0.0`
  - well adapted to OSS components
  - just added to ECMA (European Computer Manufacturers Association)
  - e.g. `pkg:npm/@babel/core@7.21.0` `pkg:golang/golang.org/x/text@0.3.7`
- CPE (Common Platform Enumeration):
  - `cpe:2.3:a:vmware:spring_cloud_bindings:2.0.0:*:*:*:*:*:*:*`
  - allowed automation against NVD queries
  - maintained by a small group of people, centralized
  - e.g. `cpe:2.3:a:babel:core:7.21.0:*` `cpe:2.3:a:golang:x.text:0.3.7:*`

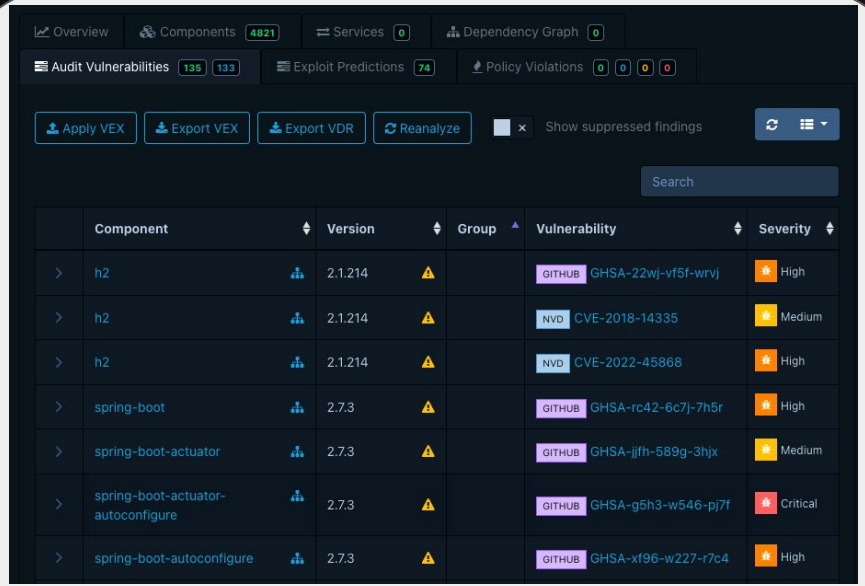
What are the tools to generate SBOMs?

- Syft:
  - OSS
  - scan libraries and images
  - formats: CycloneDX, Syft, SPDX
- Maven CycloneDX plugin:
  - OSS
  - directly from your build
- Trivy, Snyk CLI, other vendors
- Your favorite Framework! Spring Boot >= 3.3 / Quarkus >=3.14.3

We have an SBOM! Let's put it to work!

my-app.jar
 l-MyApp.class
 l-spring.jar
Ubuntu
 l-openssl

SBOM.json

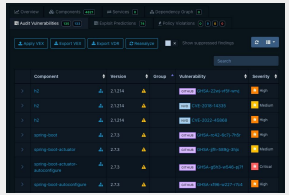


Software Component Analysis (SCA) Dashboard

Database(s) of vulnerabilities?

The life of a new Vulnerability

SBOM.json



SCA Dashboard

Use your own CVE Numbering Authority (CNA) and submit to CVE,org

Go to the project Github Advisory
OR email the author,
OR tell the package registries,
OR submit via bounty programs

No answer?
Submit to NVD after 30 days

R
A
T
I
N
G

CVSS

NVD

GHSA

OSV

Local advisories

OS dependencies vulnerabilities

- You not only deploy a jar (or war or… ear !) but a Docker (OCI) image too!
- If you own the Dockerfile or the platform - you're responsible for them 😱
- Hopefully, you can get rid of them
  - Do you need curl, wget, bash in your image?
- Even better: distroless base images
  - Docker Hardened Images
  - Chainguard Wolfi
  - Rapidfort, Minimus, Bellsoft Alpaquita
- … or ask your vendor to fix the CVEs for you!

Java libraries dependencies vulnerabilities

- Stay on the latest versions!
  - Try and automate your migration with OpenRewrite / Renovate

Given a version number MAJOR.MINOR.PATCH, increment the:

⚠️⚠️⚠️

3. PATCH version when you make backward compatible bug fixes ⚠️ ⚠️

https://semver.org/

  - Regularly check the migration guides for Spring Boot / Quarkus
  - Avoid overriding versions yourself…

How to deal with remaining vulnerabilities

- Beware of false positives!
  - Check your SBOMs - all components properly detected?
  - Use custom policies to get rid of them
- Share and re use your triage efforts
  - Software Vendor? Create and share VEX files!
  - Software Customer? Ask for VEX files!

Conclusion: towards 0 CVEs?

- Start integrating SAST in your pipeline, any tool will do!
  - Review and enable all Github/Gitlab security features
- Start generating SBOMs, even if you don't vuln-check them yet!
- Use distroless base images as much as you can!
- Migrate before your framework minor is EOL!
- Share your Vex files!

https://linktr.ee/nocve

📸 presentation content and references 📸